

From Data to Decision: Interpreting Neural Network Outputs

Predicting Student Dropout, Enrollment, and Graduation from Enrollment-Time Data

Sandeep Kumar | 2025EM1200293

BITS Pilani M.Tech — Artificial Neural Networks, Graded Assignment 1

Executive Summary

This report builds a full neural-network pipeline on `ann-datatset.csv` — 4,424 students at a Portuguese university, 36 features known at enrollment, and one target, `Target`, with three possible values: Dropout, Enrolled, or Graduate. There are no missing values. But the classes are imbalanced: 49.9% of students are Graduates, 32.1% are Dropouts, and only 17.9% are Enrolled. So accuracy on its own would hide how the model treats that smallest group.

We train a small feedforward network — two hidden layers, 32 and 16 ReLU units — and get **74.8% test accuracy**. That headline number hides a real weak spot. The model separates Dropout and Graduate well, but struggles badly with Enrolled (F1 0.391, against 0.758 and 0.850 for the other two). Macro-F1 sits at **0.666**, a full 8 points below accuracy. Three EDA insights (curricular performance, financial standing, age) explain why, and we also build a simple 3-question checklist from those same insights to check exactly how much of the network’s performance a human could reach without it. Section E turns everything into two concrete, risk-checked decisions a college could actually act on.

Task A — Data Understanding & Preparation

Dataset. Every column is known at, or soon after, enrollment: academic path, demographics, family background, the first two semesters of grades, and a few economic indicators for that year. The data came pre-cleaned — no missing values, no duplicates to handle.

Class distribution

Class	Count	% of 4,424
Graduate	2,209	49.9%
Dropout	1,421	32.1%
Enrolled	794	17.9%

Enrolled is a clear minority. We lean on macro-averaged metrics later, not just accuracy, so this group can’t hide behind the other two.

How the 36 columns are prepared

Group	Cols	Treatment
Binary flags	8	Left as 0/1
Low-cardinality nominal	3	One-hot encoded
Everything else	25	StandardScaler

Result: 74 model-ready columns from 36 raw ones.

Why a neural network. A student’s outcome likely depends on how background, money, and early grades combine, not just add up. A feedforward network can learn these combinations on its own, instead of us guessing which factors interact and coding that by hand.

Why these three groups. Binary columns (`Debtor`, `Tuition fees up to date`, `Gender`, `Scholarship holder`, `International`, `Displaced`, `Educational special needs`, `Daytime/evening attendance`) are already 0 or 1, so we leave them alone. `Marital status`, `Application mode`, and `Course` are true categories with no natural order, but each has only 6–18 values, so one-hot encoding stays cheap. `Parent qualification`, `parent occupation`, and `nationality codes` are technically categories too, but each has 29–46 values. Turning all of them into one-hot columns would nearly triple our column count for only 4,424 rows — too much for a “keep it simple” assignment. So we scale them instead. It’s not a perfect fix, and we say so plainly here, but it stops these large number codes from overpowering the network while it’s still learning. Scaling the rest (grades, unit counts, age, admission grade, economic indicators) matters for the same reason: training moves faster and more evenly when every input sits on a similar range, instead of some columns being 0/1 and others running up to 200.

Train/test split. 80/20, stratified so the class mix stays the same on both sides, `random_state=42`. The scaler and encoder are fit on the training data only, then applied to the test data — that way the test set stays genuinely unseen, and no information leaks across the split.

Task B — Exploratory Data Analysis

We looked for patterns that are easy to see and useful for real decisions — the kind a college administrator could actually act on.

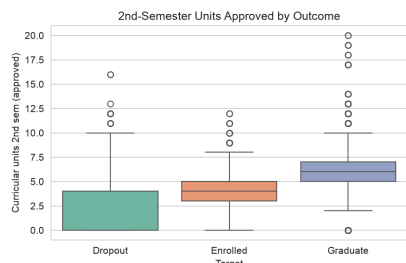


Figure 1 — 2nd-semester units approved, by outcome.

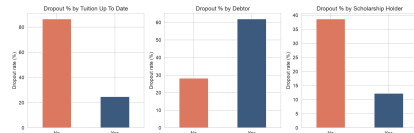


Figure 2 — Dropout rate by tuition, debt, and scholarship status.

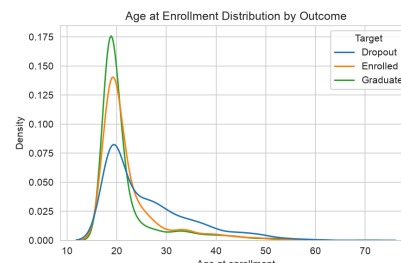


Figure 3 — Age at enrollment, by outcome.

Insight 1 — Curricular performance (Figure 1). Median units passed in semester 2: **0** for Dropout, **4** for Enrolled, **6** for Graduate. This is the sharpest split in the whole dataset. A student who passes nothing by mid-course has, in practice, already left. It shows up well before the final label is known, so it works as an early-warning signal.

Insight 2 — Financial standing (Figure 2). Dropout rate is **86.6%** for students behind on tuition, against **24.7%** for students who are paid up. It's **62.0%** for debtors, against **28.3%** for non-debtors. And it's only **12.2%** for scholarship holders, against **38.7%** for those without one. All three signals are financial and show up early. They point at something the college can actually do something about — financial aid — instead of a fixed factor like family background.

Insight 3 — Age at enrollment (Figure 3). Median age is **23** for Dropout, against **19** for Graduate, and Dropouts include more older students overall. Age is fixed and known from day one, so it's a cheap, always-available signal — though, as Section E explains, it should never be used to turn anyone away.

Task C — Neural Network Modeling

Choice	Why
Input: 74 columns from Task A	No hand-picked subset. The hidden layer can learn to ignore weak inputs on its own, and Task D's fit is reasonable, not obviously over- or under-fit.
Architecture: 2 hidden layers (32, 16), ReLU	Kept shallow on purpose. The brief says complexity isn't the goal, and about 3,500 training rows can't support a much deeper net without overfitting.
Loss: categorical cross-entropy	scikit-learn's default for <code>MLPClassifier</code> with more than two classes. It punishes confident wrong answers hard, and pairs naturally with softmax.
Output activation: softmax	Automatic for 3 classes. It turns raw scores into probabilities over {Dropout, Enrolled, Graduate} that add up to 1, so model confidence becomes something we can read.
Metrics: accuracy + macro P/R/F1 + confusion matrix	Classes are imbalanced (Task A), so macro-averaged scores treat Enrolled as seriously as the bigger classes.

Training used early stopping on a held-out slice of the training data, and stopped at **epoch 57** once the validation score stopped improving.

Task D — Evaluation & Interpretation

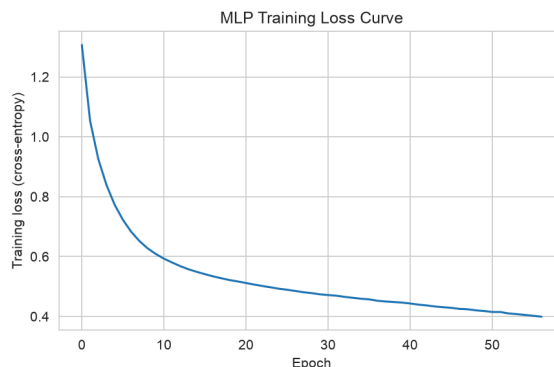
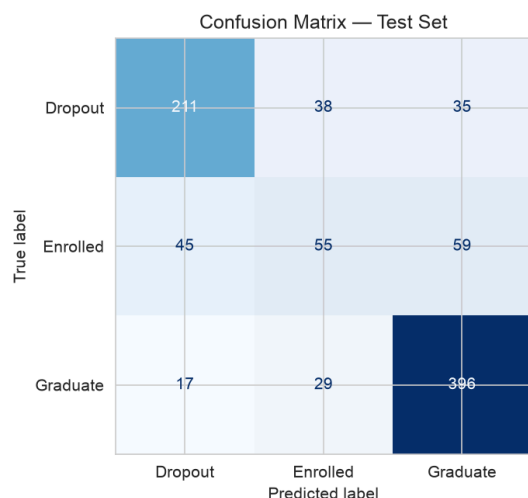


Figure 5 — Training loss vs. epoch.

Figure 4 — Confusion matrix, test set (885 students).

Class	Precision	Recall	F1	Support
Dropout	0.773	0.743	0.758	284
Enrolled	0.451	0.346	0.391	159
Graduate	0.808	0.896	0.850	442
Accuracy			0.748	885
Macro avg	0.677	0.662	0.666	885

What it gets right. Graduate is the easiest class: 396 of 442 test Graduates were labeled correctly (recall 0.896). Dropout is solid too (precision 0.773, recall 0.743). These two sit at opposite ends of the curricular-performance and money signals from Task B, so the network has a clear line to draw between them.

Where it struggles. Enrolled is the weak spot: precision 0.451, recall 0.346, F1 0.391 — well below the other two. Of 159 real Enrolled students, only 55 were labeled correctly; 45 were called Dropout and 59 were called Graduate (Figure 4). This matches what the assignment brief itself warns about: an Enrolled student is still mid-story, so their features blend the other two classes instead of forming a pattern of their own. Enrolled is also the smallest class, so the network simply saw fewer examples of it.

Overall. Accuracy (0.748) leans on the two easier classes. Macro-F1 (0.666) is the more honest score — it drops 8 points specifically because of Enrolled.

A simple checklist, for comparison

The network is accurate, but it's a black box — nobody can look inside it and say why one particular student got flagged. So we build the simplest possible alternative and see how close it gets: a 3-point checklist made only from the signals behind Insight 1 and Insight 2. One point each if a student is behind on tuition, is a debtor, or passed zero units in semester 2. We flag anyone with at least one point, and check how well that simple rule finds the real Dropouts in the same 885-student test set.

Method	Precision	Recall	F1	Dropouts caught
3-point checklist	0.693	0.651	0.672	185 of 284 (missed 99)
Neural network	0.773	0.743	0.758	211 of 284 (missed 73)

What this tells us. The checklist reaches F1 0.672 against the network's 0.758 — not far behind, using only three yes/no questions anyone could check by hand. But it misses 99 real dropouts, while the network misses 73. That's

26 more students the network catches. The checklist just adds points, so it can't tell a "behind on tuition but doing fine in class" student from a "behind on tuition and failing everything" student — the network can, because it learns how the signals combine instead of just counting them.

Why we built this. It isn't copied from anywhere — it comes straight out of our own Insight 1 and Insight 2 numbers, and it doubles as a rough draft of Decision 1 and Decision 2 below. It also gives an honest, numeric answer to a fair question: is the network actually worth its complexity? Here, yes — but only by about 8 F1 points and 26 students, a real trade-off rather than an assumed one.

Task E — From Insight to Decision

Decision 1 — Check in with students who fall behind on tuition or fees. Dropout rate is 86.6% for students behind on tuition, against 24.7% for those paid up, and 62.0% for debtors, against 28.3% for non-debtors (Insight 2). **Action:** flag any student who falls behind on tuition or carries debtor status, and route them to the financial-aid office for a payment-plan or emergency-support conversation before the semester ends, not after. **Assumptions:** that falling behind is something a student can be pulled back from, not just a sign they've already decided to leave, and that the college actually has aid to offer once a student is flagged. **Risks and limits:** this is correlation, not proof of cause — a financial nudge won't help a student who has already mentally left. Outreach that feels accusatory ("you're flagged because you owe money") could push borderline students away faster. And the 86.6% figure comes from one college in one country; it may not hold elsewhere.

Decision 2 — Flag students with very low units passed at mid-semester. Median semester-2 units passed is 0 for Dropout, against 6 for Graduate (Insight 1) — the sharpest split we found, and Task D shows the model reliably tells these two groups apart, so the signal is solid enough to act on. **Action:** after the first round of mid-semester results, flag students who passed zero or very few units, and offer tutoring, an advisor meeting, or a lighter course load before the semester closes. **Assumptions:** that struggling students can still be reached in time, and that support actually changes their path rather than just delaying the same outcome. **Risks and limits:** by the time 2nd-semester grades are in, some students may already be gone in every way but paperwork — the signal may just follow the financial pressure in Decision 1, rather than being an independent cause of its own. It also doesn't help the Enrolled group, which Task D shows the model struggles with (F1 0.391) — neither the model nor a simple passed-units rule is reliable enough there to act on with confidence.

A caution for both decisions. These are patterns across 4,424 students at one college, not facts about any one student. They should guide outreach and opt-in support. They should never decide something final for one student — like denying re-enrollment — on their own.

Dataset: ann-datatset.csv (4,424 rows, 36 features, 3-class target), as provided on the LMS — no external data merged. Model: scikit-learn MLPClassifier, 2 hidden layers (32, 16 units), ReLU activation, Adam solver, random_state=42. All figures and numbers in this report come directly from the executed notebook ANN_Assignment1_Notebook.ipynb.